Software Testing

ASSIGNMENT 2

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.

S3	**Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.**
S4	**Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.**

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2    return INVALID
 3  rate = 40
 4  if zone == "B" then
 5    rate = 60
 6  else if zone == "C" then
 7    rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12    w = w - 1
13  if express then
14    if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**

- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**

- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**
- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

- **Describe the new and changed test cases**
- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

- **Equivalence classes and boundaries**

- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**
- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

MARK DISTRIBUTION

Task	Marks

Task 1 – Select Test Case Design Strategies	**5**
Task 2 – Evaluate Random Testing	**2**
Task 3 – Design and Evaluate Equivalence Classes	**6**
Task 4 – Design and Evaluate Boundary Value Tests	**4**
Task 5 – Apply Other Black-Box Techniques	**3**
Task 6 – Design and Evaluate State-Transition Tests	**3**
Task 7 – Construct the Control Flow Graph	**4**
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	**4**
Task 9 – Judge Test Adequacy and Coverage	**3**
Task 10 – Compare Black-Box and White-Box Effectiveness	**4**
Task 11 – Recommend and Defend the Final Test Suite	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

ANSWERS

TASK 1 – Select Test Case Design Strategies (5 Marks)

Ans:

Position: I use three different strategies, one per kind of artefact, and combine them. No single strategy finds every kind of defect in SwiftShip.

Black-box vs white-box: black-box design derives tests from the specification and ignores how the code is written. White-box design derives tests from the code's structure (statements, branches, paths).

Table 1: Strategy selection for SwiftShip

Part	Strategy selected	Information needed	Likely to reveal	Unlikely to reveal
Charge rules S1–S4	Equivalence classes + boundary values + decision table	Specification only	Missing validation (invalid zone), wrong thresholds (30/30.1, 50/50.1, 10), wrong rule combinations	Dead code, code-level weaknesses (floating-point money)
shipCharge pseudocode	Control flow graph, basis-path testing, branch/condition coverage	The code	Untested code, unreachable paths, loop and structure problems; measures how much code the tests exercise	Missing code (omissions): you cannot cover code that is not written
Tracking states	State-transition testing	State model and transition rules	Illegal transitions allowed, retry-limit and cancellation faults	Calculation errors

Supplement	Random testing	Input ranges	Crashes and robustness failures	Specific boundaries, expected values

Why combine: the defects have different causes.

- The invalid-zone defect is a missing branch. Only a spec-derived test finds it, because code-based coverage never asks for it.
- Whether the loop and decisions have been exercised can only be measured from the code.
- Tracking has no numeric ranges, so ECP/BVA do not apply; only a state model reveals illegal transitions.
- Black-box finds what the code omits, white-box shows what the tests never executed, and the state model covers the module the other two cannot.

Effort judgement: the combination costs about 35 automated tests (Task 11), far below exhaustive testing and far more targeted than random testing.

TASK 2 – Evaluate Random Testing (2 Marks)

Ans:

Position: random testing cannot replace systematic design for shipCharge. It is a useful extra check for crashes only.

How it would be done: generate 1,000 inputs. Weight is chosen uniformly from 0.0–60.0 in steps of 0.1 (601 values). Zone is chosen from A, B, C or other, express is Yes or No, and parcels from 0–20. Each result is compared with an expected value.

Probability of hitting important values (1,000 tests)

Target	**Probability per test**	**Chance it appears at least once**

A single weight, e.g. 50.0 or 30.1 (alone)	1/601 = 0.17%	1 − (600/601)^1000 ≈ **81%**
30.0 or 30.1 **with express = Yes**	1/1202	≈ **56%**
All six key weights (0, 0.1, 30, 30.1, 50, 50.1) with express on 30/30.1	–	≈ $0.81^4 \times 0.56^2$ ≈ **14%**
Weight 30.0 + express + parcels 10 + zone C	1/(601×2×21×4) ≈ 1/1,00,000	≈ **1%**

About 10 designed tests hit all the key boundaries with certainty. If weights are drawn as real numbers rather than 0.1 steps, the probability of landing exactly on a boundary is effectively zero.

Oracle problem: random inputs need an independently computed expected result.

- Calculating 1,000 results by hand is impractical.
- If the expected value comes from the code itself, defects are copied into the oracle.
- If it comes from a second implementation of the spec, a disagreement tells you something is wrong but not which side, and the second implementation needs its own testing.

Judgement: the generator would find the invalid-zone defect only if it deliberately produces "other" zones, and it would still need an oracle to recognise the wrong answer. Random testing is useful for crashes, since no oracle is needed, but it cannot replace designed tests. Use it only as a supplement.

TASK 3 – Design and Evaluate Equivalence Classes (6 Marks)

Ans:

How classes were identified: each threshold in the spec splits a range. S1 gives thresholds 0 and 50. S3 adds 30 inside the valid range. S2 adds the whole-kg versus fractional-kg distinction (started-kilogram rule) and the "any other zone" class. S4 gives the threshold 10.

Table 2: Equivalence classes

ID	Input	Class	Valid / Invalid	Source
W1	Weight	$0 < w \leq 30$, whole number	Valid	S1, S2
W2	Weight	$0 < w \leq 30$, with a fraction (started kg)	Valid	S2
W3	Weight	$30 < w \leq 50$	Valid	S1, S3
W4	Weight	$w \leq 0$	Invalid	S1
W5	Weight	$w > 50$	Invalid	S1
Z1 / Z2 / Z3	Zone	A / B / C	Valid	S2
Z4	Zone	any other value	Invalid	S2
E1 / E2	Express	Yes / No	Valid	S3
E3	Express + weight	Yes with $w > 30$ → NOT_ALLOWED	Valid input, special outcome	S3
P1	Parcels	0–9 (no discount)	Valid	S4
P2	Parcels	≥ 10 (10% discount)	Valid	S4
P3	Parcels	negative (assumption A2)	Invalid	–

Table 3: Test cases and expected results

ID	Weight	Zone	Express	Parcels	Classes covered	Expected result
T1	2.5	A	No	5	W2, Z1, E2, P1	3 kg × 40 = **₹120**
T2	10	B	Yes	12	W1, Z2, E1, P2	10×60 = 600 ×1.5 = 900 ×0.9 = **₹810**
T3	45	C	No	3	W3, Z3, E2, P1	45×90 = **₹4,050**

T4	35	A	Yes	2	E3	**NOT_ALLOWED**
T5	−5	A	No	1	W4	**INVALID**
T6	60	A	No	1	W5	**INVALID**
T7	4	“D”	No	1	Z4	**INVALID**
T8	4	A	No	−1	P3	**Rejected** (assumption, spec to confirm)

Why this is the smallest set: every valid class is covered by T1–T3. A set cannot be smaller than the largest number of classes in one input, which is 3 for weight and 3 for zone. T4 is needed because the NOT_ALLOWED outcome is a class of its own. Invalid classes are tested **one at a time**. If two invalid values are combined, the weight check on line 1 fires first and hides the zone defect.

Effectiveness evaluation

- **Fails against the pseudocode: T7**. The code has no zone validation. Any unknown zone falls through to rate 40, so zone “D” with 4 kg returns ₹160 instead of INVALID. T8 would also fail if the hospital-style clarification confirms that negative parcels must be rejected.
- **Passes:** T1–T6 pass, so rate selection, the started-kilogram loop, express, discount and the weight limits work.
- **What this reveals:** the defect is an omission, found only because the invalid class came from the spec and not from the code.
- **Limit of ECP:** one value per class can miss an edge error such as >= instead of > at 50 or 30. That is why Task 4 is needed.

TASK 4 – Design and Evaluate Boundary Value Tests (4 Marks)

Ans:

Most defect-prone boundaries: 0/0.1 and 50/50.1 (S1), 30/30.1 (express limit, S3), the started-kilogram steps such as 1.0/1.1 (S2) and parcels 9/10 (S4). They are where > versus >= mistakes happen.

Table 4: Forms of BVA (weight and parcels are the two ordered inputs, so n = 2)

Form	Rule	Tests for n = 2	Comment
Normal BVA	min, min+, nominal, max−, max, 4n+1	9	No invalid-side values
Robust BVA	adds min−, max+, 6n+1	13	Tests rejection at 0 and 50.1
Worst-case BVA	5^n	25	Every combination of boundaries
Robust worst-case	7^n	49	Heavy and mostly redundant

Zone and express are not ordered, so BVA does not apply to them.

Table 5: Boundary tests selected (zone A, express No, parcels 0 unless stated)

ID	Input	Expected
B1	weight 0	INVALID
B2	weight 0.1	1 kg → ₹40
B3	weight 1.0	1 kg → ₹40
B4	weight 1.1	2 kg → ₹80
B5	weight 30.0, express Yes	30×40 = 1,200 ×1.5 = **₹1,800**
B6	weight 30.1, express Yes	NOT_ALLOWED
B7	weight 50.0	50×40 = **₹2,000**
B8	weight 50.1	INVALID
B9	weight 1.0, parcels 9	₹40 (no discount)
(parcels 10)	weight 1.0, parcels 10	40×0.9 = ₹36 (covered by D2 in Task 5)

Recommended form: robust BVA on weight (it needs both invalid-side values 0 and 50.1), plus the internal boundaries 30/30.1 and 1.0/1.1, and the two parcels values 9/10. That is about 10 tests. Worst-case BVA (25–49 tests) is not worth it. Weight changes the base charge and parcels only applies a multiplier, so their interaction is covered by the decision table (Task 5) at far lower cost. Robust BVA is also needed for the risk: money errors at boundaries affect 40,000 parcels a day.

TASK 5 – Apply Other Black-Box Techniques (3 Marks)

Ans:

Decision table for S3 and S4 (valid weight and zone assumed)

Rule	**Express?**	**Weight ≤ 30?**	**Parcels ≥ 10?**	**Outcome**
R1	N	Y	N	Base charge
R2	N	Y	Y	×0.9
R3	N	N	N	Base charge
R4	N	N	Y	×0.9
R5	Y	Y	N	×1.5
R6	Y	Y	Y	×1.5 ×0.9 = ×1.35
R7	Y	N	N	NOT_ALLOWED
R8	Y	N	Y	NOT_ALLOWED

Collapsing "don't care" conditions gives 5 reduced rules (weight is irrelevant when express is No, and parcels is irrelevant when express is Yes with weight > 30). All 8 are cheap, so I run all 8.

Tests (zone A, rate ₹40/kg)

ID	**Rule**	**Input**	**Expected**
D1	R1	10 kg, N, parcels 5	₹400

D2	R2	10 kg, N, parcels 10	₹360
D3	R3	35 kg, N, parcels 5	₹1,400
D4	R4	35 kg, N, parcels 10	₹1,260
D5	R5	10 kg, Y, parcels 5	₹600
D6	R6	10 kg, Y, parcels 10	₹540
D7	R7	35 kg, Y, parcels 5	NOT_ALLOWED
D8	R8	35 kg, Y, parcels 10	NOT_ALLOWED

Comparison with the ECP/BVA set

- T1–T8 and B1–B9 cover only **R1, R2, R3, R5, R6, R7**, which is 6 of 8.
- **Not covered: R4** (heavy parcel with discount, no express) and **R8** (frequent customer requesting express above 30 kg).
- **Why it matters:** R6 checks that the discount is applied to the *final* charge. R8 checks that NOT_ALLOWED is returned and not a discounted charge. These interaction faults produce no wrong output in single-rule tests.
- **Conclusion:** ECP and BVA do miss combination defects. Two extra tests (D4, D8) close the gap.

TASK 6 – Design and Evaluate State-Transition Tests (3 Marks)

Ans:

State-transition table (8 valid transitions; every other state/event is invalid)

#	From	Event	To	Rule
1	BOOKED	pickup	PICKED_UP	–

2	BOOKED	cancel	CANCELLED	Only before pickup
3	PICKED_UP	scan	IN_TRANSIT	–
4	IN_TRANSIT	dispatch	OUT_FOR_DELIVERY	–
5	OUT_FOR_DELIVERY	deliver	DELIVERED	–
6	OUT_FOR_DELIVERY	fail	FAILED_ATTEMPT	–
7	FAILED_ATTEMPT	retry	OUT_FOR_DELIVERY	Max 2 retries
8	FAILED_ATTEMPT	return	RETURNED	–

DELIVERED, RETURNED and CANCELLED are end states with no outgoing transitions.

Valid-transition tests (cover all 8 transitions)

ID	Sequence
SV1	BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED
SV2	BOOKED → CANCELLED
SV3	…OUT_FOR_DELIVERY → FAILED → OUT_FOR_DELIVERY (retry 1) → FAILED → RETURNED
SV4	OUT → FAILED (1) → retry 1 → FAILED (2) → retry 2 → FAILED (3) → **third retry must be rejected**; only RETURNED allowed

Table: invalid transitions selected and why (56 of the 64 state × event cells are invalid, so I choose by business harm)

ID	Invalid transition	Harm if allowed
SI1	PICKED_UP → CANCELLED	Cancellation rule boundary: refund for a parcel already collected

SI2	OUT_FOR_DELIVERY → CANCELLED	Refund while the parcel is on the van
SI3	BOOKED → DELIVERED	False delivery confirmation, reputational damage, wrong claims
SI4	FAILED_ATTEMPT → DELIVERED	Unconfirmed delivery after a failed attempt
SI5	DELIVERED → RETURNED	Customer-refund fraud
SI6	DELIVERED → OUT_FOR_DELIVERY	Re-dispatch of a delivered parcel, wasted cost
SI7	CANCELLED → PICKED_UP	Courier trip for a cancelled booking
SI8	IN_TRANSIT → PICKED_UP	Backward move corrupts tracking history

The retry-limit test is SV4. The cancellation rule is tested positively (SV2) and negatively (SI1, SI2). Total: 4 valid + 8 invalid = **12 tests**.

TASK 7 – Construct the Control Flow Graph (4 Marks)

Ans:

Table: node–line mapping (line 1 is a compound condition, split into two decision nodes because of short-circuit evaluation)

Node	**Line(s)**	**Content**	**Type**
N1	1	weight ≤ 0	Decision
N2	1	weight > 50	Decision
N3	2	return INVALID	Exit
N4	3	rate = 40	Process

N5	4	zone == “B”	Decision
N6	5	rate = 60	Process
N7	6	zone == “C”	Decision
N8	7	rate = 90	Process
N9	8–9	charge = 0; w = weight	Process
N10	10	w > 0	Decision (loop)
N11	11–12	charge += rate; w −= 1	Process
N12	13	express	Decision
N13	14	weight > 30	Decision
N14	15	return NOT_ALLOWED	Exit
N15	16	charge ×= 1.5	Process
N16	17	parcels ≥ 10	Decision
N17	18	charge ×= 0.9	Process
N18	19	return charge	Exit

Edges (23, plus 3 edges to a virtual exit X = 26):

N1→N3 (T), N1→N2 (F), N2→N3 (T), N2→N4 (F), N4→N5, N5→N6 (T), N5→N7 (F), N6→N9, N7→N8 (T), N7→N9 (F), N8→N9, N9→N10, N10→N11 (T), N10→N12 (F), N11→N10, N12→N13 (T), N12→N16 (F), N13→N14 (T), N13→N15 (F), N15→N16, N16→N17 (T), N16→N18 (F), N17→N18, N3→X, N14→X, N18→X.

Decision nodes: N1, N2, N5, N7, N10, N12, N13, N16 (8 in all). The loop is N10 with back edge N11→N10. The compound condition at line 1 is two decisions, one per condition.

Does the code structure match the specification?

Spec rule	Code	Match?
S1 invalid weight	Lines 1–2, two decisions	✓ Matches
S2 zones: A, B, C, else INVALID	Lines 3–7 give three rates; no node returns INVALID	✗ **Mismatch:** the spec has 4 zone outcomes, the code only 3, so an unknown zone silently becomes Zone A
S2 started kilogram	Loop N10–N11 runs ⌈weight⌉ times	✓ Correct, but structurally different (the spec has no loop)
S3 express limit	Lines 13–16	✓ Correct, but the charge is calculated first and thrown away when NOT_ALLOWED (wasted loop of up to 50 iterations)
S4 discount	Line 17	✓ Matches

The graph exposes the missing INVALID branch for zones: the CFG has no path that the spec's fourth zone outcome could take.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity (4 Marks)

Ans:

Cyclomatic complexity: three methods

1. **E − N + 2:** with the virtual exit, E = 26 and N = 19, so V(G) = 26 − 19 + 2 = **9**.
2. **Predicates + 1:** 8 simple decisions (N1, N2, N5, N7, N10, N12, N13, N16), so 8 + 1 = **9**.
3. **Regions:** the graph has 8 enclosed regions plus the outer one, giving **9**.

If line 1 were treated as a single decision, the value would be 8. Because short-circuit evaluation creates two branches, I use 9.

Table: basis paths and test data

Path	Route	Test data (weight, zone, express, parcels)	Expected
P1	Baseline: N1, N2, N4, N5, N7, N9, N10, N11, N10, N12, N16, N18	1.0, A, No, 5	₹40
P2	N1 → N3	0, A, No, 5	INVALID
P3	N1, N2 → N3	50.1, A, No, 5	INVALID
P4	Zone B (N5 → N6)	1.0, B, No, 5	₹60
P5	Zone C (N7 → N8)	1.0, C, No, 5	₹90
P6	Loop executed several times	2.5, A, No, 5	3 kg → ₹120
P7	N12, N13, N15 (express allowed)	1.0, A, Yes, 5	₹60
P8	N12, N13 → N14	35, A, Yes, 5	NOT_ALLOWED
P9	N16 → N17 (discount)	1.0, A, No, 10	₹36

Each path adds at least one edge not used by earlier ones, apart from P6, which is independent because it repeats the loop edge. So the 9 paths are linearly independent.

Infeasible cases

- **Zero loop iterations:** line 1 guarantees weight > 0, so the loop body always runs at least once. This is why P6 replaces the usual "skip the loop" path.
- **Line 1, both conditions true** (weight ≤ 0 and weight > 50): impossible.

Loop testing (the loop runs ⌈weight⌉ times, so 1 to 50 iterations)

Iterations	Weight	Expected (zone A, no express)
0	not possible	–
1	0.1	₹40

2	1.1	₹80
Typical	10	₹400
Max − 1 (49)	49.0	₹1,960
Max (50)	50.0	₹2,000

Is 9 enough? No. The basis paths do not test the boundaries 30/30.1 and 50/50.1, do not include an invalid zone, and do not exercise the loop at its maximum. Counting each iteration count as a distinct path, there are about **542 feasible complete paths** (iterations 1–30: 30 × 3 zones × 4 outcomes = 360; iterations 31–50: 20 × 3 × 3 = 180; plus 2 invalid paths), which is impractical to test. Nine basis tests are a minimum, not a sufficient suite.

TASK 9 – Judge Test Adequacy and Coverage (3 Marks)

Ans:

Table: coverage criteria for shipCharge

Criterion	**Meaning here**	**Minimum tests**	**Achieved by the 9 basis tests?**
Statement	Every statement runs	3	✓ 100%
Branch (decision)	All 16 decision outcomes	5	✓ 100%
Condition	Each atomic condition true and false	5	✓ 100%
Multiple condition (line 1: 4 combinations)	All combinations	3 feasible	✓ 100% of feasible (T,T is impossible)
MC/DC (line 1)	Each condition independently changes the outcome	–	✓ (weight 0 vs 1.0 and 50.1 vs 1.0)

Minimum sets: three tests give statement coverage (for example weight 0; weight 10, zone B, express, parcels 12; weight 35, zone C, express). Five give branch coverage (weight 0; 60; 10/B/Yes/12; 35/C/Yes/3; 2/A/No/3).

Next stronger criterion: the basis tests already reach MC/DC, so the next step is path coverage, with 542 paths. That is not affordable. My compromise adds only the loop-boundary tests (about 5, mostly shared with the BVA set).

Would 100% coverage have found the invalid-zone defect? No.

- The defect is missing code. Every statement and branch can be fully covered using only zones A, B and C.
- A test with zone "D" takes the same path as Zone A (N5 false, N7 false), which is already covered, so a coverage tool sees nothing new.
- Even if "D" is run, nothing flags the ₹160 result as wrong without an expected value from the spec.

Recommended criterion: 100% branch coverage plus MC/DC at line 1 (cheap, reached by the basis set), **combined with 100% requirement coverage** (all ECP classes, all 8 decision-table rules, all boundaries). Defence: this module calculates money for 40,000 parcels a day, so the cost of a wrong rule is high, and branch coverage costs only 5–9 tests. Path coverage costs about 542 tests and buys little extra. A single coverage number is dangerous because it says nothing about omissions.

TASK 10 – Compare Black-Box and White-Box Effectiveness (4 Marks)

Ans:

Table: defects and observations found

#	Defect / issue	Black-box	White-box

D1	Invalid zone accepted, charged as Zone A (High)	✓ Found (T7)	✗ Missed by coverage tests; visible in the CFG review only as a missing branch
D2	No validation of negative parcels or extra decimals (Medium, spec gap)	✓ Found (T8, once clarified)	✗ Missed (missing code)
D3	Money is calculated in binary floating point, no rounding rule (Low–Medium)	✗	✓ Seen by code reading
D4	Express limit checked after the loop, wasteful (Low)	✗ Output is correct	✓ Seen in the CFG
–	Defects found by **both**	None in the current code. Both confirm the boundaries, loop and discount are correct	

Seeded-fault check (my own mutation exercise: six deliberate code changes, to see which test sets detect them)

Mutant	**ECP (T1–T8)**	**BVA (B1–B9)**	**Basis (P1–P9)**
M1: > 50 → >= 50	✗	✓ (B7)	✗
M2: > 30 → >= 30	✗	✓ (B5)	✗
M3: >= 10 → > 10	✗	✓ (D2/B-set parcels 10)	✓ (P9)
M4: w > 0 → w >= 1 (started kg)	✓ (T1)	✓ (B4)	✓ (P6)
M5: * 1.5 → * 1.4	✓	✓	✓
M6: missing INVALID for zones	✓ (T7)	✗	✗

Detected	3 of 6	5 of 6	3 of 6

ECP + BVA together detect all 6. Adding the basis tests detects nothing more.

Interpretation

- Black-box found the important defect (D1) and all boundary mutants. White-box *tests* found nothing beyond that, because the code's boundaries are correct and its only serious defect is an omission.
- Black-box missed D3 and D4 because they produce correct outputs. White-box *reading* found them.
- **Removable overlap:** all nine basis tests duplicate black-box tests. For example P1 ≡ B3, P2 ≡ B1, P3 ≡ B8, P5 ≡ T3, P6 ≡ T1, P7 ≡ D5, P8 ≡ D7, P9 ≡ D2. White-box is best used to measure coverage of the black-box suite and to expose untested code, not to add more tests.

Would this hold for other modules? Not necessarily. This is one 19-line function with one real defect. Modules with complex hidden logic (fraud rules, address parsing) would show more white-box-only defects. The tracking module is driven by states, so neither ECP nor path testing suits it; state-transition tests are the effective approach there.

TASK 11 – Recommend and Defend the Final Test Suite (4 Marks)

Ans:

Table: final suite (23 calculation tests + 12 tracking tests = 35)

Group	Tests	Technique	Count
Class representatives	T1, T2, T3, T7, T8	ECP	5
Boundaries	B1–B9	Robust BVA (weight), boundary at parcels 9	9
Rule combinations	D1–D8	Decision table	8

Loop extreme	L4 (49.0 kg → ₹1,960)	Loop testing	1
Tracking valid	SV1–SV4	State transition	4
Tracking invalid	SI1–SI8	State transition (risk-based)	8
Total			**35**

Removed as duplicates (17 of 40 calculation tests): T4, T5, T6 (same classes as D7, B1, B8), B10 (same as D2), L1, L2, L3, L5 (same as B2, B4, D1, B7), and P1–P9 (all covered by the black-box tests).

Why this beats the alternatives

- vs **random testing**: deterministic, readable and reaches every boundary.
- vs **9 basis paths alone**: detects 6 of 6 seeded faults instead of 3 of 6.
- vs **worst-case BVA (25–49 tests) or a full cross-product (≈96)**: the extra combinations add no new faults, since zone only changes the rate (covered by T2 and T3).

Coverage achieved: 100% statement, 100% branch, MC/DC at line 1, all 9 basis paths, all ECP classes, 8 of 8 decision-table rules, all weight boundaries and the parcels 9/10 boundary, and 8 of 8 valid tracking transitions.

Effort: the calculation tests run in milliseconds. The tracking tests take about 1–2 minutes with a database. The whole suite runs in **about 3 minutes** when automated.

Known gaps and risks

- The invalid-zone fix and the rule for negative parcels must be decided before T7 and T8 become pass/fail.
- Weights with more than one decimal and non-numeric input are not specified or tested.
- No rounding rule exists, so floating-point effects remain a risk.
- The monthly parcel count is passed in, so how it is calculated is not tested here.
- Performance and concurrent bookings are outside this suite.

TASK 12 – Re-evaluate and Adapt the Test Suite (8 Marks)

Assumed formula (the developers added the two decisions after line 16, before the discount):

16 charge = charge * 1.5 (if express)

16a if zone == "C" then charge = charge * 1.08

16b if weekend then charge = charge + 50

17 if parcels >= 10 then charge = charge * 0.9

This means the surcharge is applied after express, and the weekend fee is included in the discounted final charge. The spec owner must confirm this order (see Task 12D).

A. Impact Analysis (2 Marks)

Ans:

- **Specification:** two new rules (S5 fuel surcharge 8% on Zone C; S6 weekend delivery +₹50). A new input, weekend, is added.
- **Code:** two new decisions, adding 4 nodes and 6 edges. The graph now has N = 23 and E = 32, so V(G) = 32 − 23 + 2 = **11** (predicates 10 + 1), up from 9.
- **Equivalence classes:** new class pair for weekend (Yes/No), and Zone C now splits into "surcharge applied" versus other zones.
- **Boundaries:** the weight boundaries (0, 0.1, 30, 30.1, 50, 50.1) and parcels 9/10 do not move. But expected results change for every Zone C test, and for every weekend test.
- **Decision table:** conditions grow from 3 to 5 (express, weight ≤ 30, parcels ≥ 10, zone C, weekend). The full table has 32 rules; after collapsing don't-cares there are **17** (up from 5).
- **30-minute limit:** the suite must still run before every release. Executing 37 calculation tests by hand at 2 minutes each would take about 74 minutes, so the suite must be automated. The limit

also rules out larger designs such as worst-case BVA across the new factors (about 784 combinations).

B. Evaluation of the Original Suite (2 Marks)

Ans:

Existing tests	**Status**
Zone A and B tests with weekend = No (all of D1–D8, B1–B9, T1, T2, T7, T8, L4)	**Still valid**, provided weekend defaults to No
T3 (45 kg, Zone C)	**Needs a new expected result:** 4,050 × 1.08 = **₹4,374**
Tracking tests SV1–SV4, SI1–SI8	Unchanged
Obsolete tests	None

Not covered:

- Weekend = Yes (no existing test sets it), so 1 of the 20 branches is untested.
- Branch coverage of the old suite on the new code = **19/20 = 95%**; basis paths = **10 of 11**.
- Combinations of Zone C and weekend with express and discount (the ×1.5, ×1.08, +50, ×0.9 order is untested).
- Weekend with an express request above 30 kg.

Can the coverage criterion still be met? Yes. 100% branch and MC/DC are reachable by adding tests; no new infeasible paths appear.

C. Adaptation or Redesign (2 Marks)

Ans:

Decision: MODIFY. I do not retain the suite, because coverage drops to 95% and 12 rule combinations are untested. I do not redesign it, because the change is two local decisions: the classes, boundaries and tracking tests remain sound.

New and changed tests (derived with the decision table; weight 10 kg, zone A rate ₹40, zone C rate ₹90)

ID	Zone	Express	Weekend	Parcels	Calculation	Expected
N1	C	N	N	5	900×1.08	₹972
N2	A	N	Y	5	400+50	₹450
N3	C	N	Y	5	972+50	₹1,022
N4	C	N	N	10	972×0.9	₹874.80
N5	A	N	Y	10	450×0.9	₹405
N6	C	N	Y	10	1,022×0.9	₹919.80
N7	C	Y	N	5	900×1.5×1.08	₹1,458
N8	A	Y	Y	5	600+50	₹650
N9	C	Y	Y	5	1,458+50	₹1,508
N10	C	Y	N	10	1,458×0.9	₹1,312.20
N11	A	Y	Y	10	650×0.9	₹585
N12	C	Y	Y	10	1,508×0.9	₹1,357.20
N13	C	Y (35 kg)	Y	10	–	NOT_ALLOWED
N14	C	Y (30.0 kg)	Y	0	2,700×1.5×1.08+50	₹4,424

Together with D1, D2, D5, D6 (zone A, weekend No) these cover all 17 reduced rules. T3 gets its new expected value (₹4,374).

Techniques: decision table (N1–N13), boundary value (N14, express limit with both new rules active), and re-derived expected results for the existing Zone C test.

Effect on coverage and size

Measure	**Original suite, original code**	**Original suite, new code**	**Modified suite, new code**
Tests	35	35	**49** (+14)
V(G) / basis paths	9	11 (10 covered)	11 (11 covered)
Branch coverage	100%	95%	**100%**
Decision-table rules covered	8 of 8	4 of 17	**17 of 17**

D. Evidence-Based Defence (2 Marks)

Ans:

Evidence	**What it shows**	**Effect on decision**
Classes and boundaries	Weight and parcels boundaries unchanged; only expected values change	Keep ECP/BVA tests
Decision-table combinations	Rules grow from 5 to 17; 12 combinations untested	Add N1–N13
Cyclomatic complexity and paths	V(G) rises from 9 to 11; 10 of 11 basis paths covered	Add tests for the two new decisions
Coverage	Old suite reaches 95% branch coverage; modified suite 100%	Retain the adequacy criterion

Execution time	49 tests ≈ 3 minutes automated (calculation tests in milliseconds, tracking about 1–2 minutes); manual would take over 70 minutes	Automation is mandatory; 27 minutes of headroom remain

Reasoning: retaining the suite leaves 12 untested combinations on code that calculates money for 40,000 parcels a day. A full redesign would discard sound tests for a change of two decisions. Modifying adds 14 tests (a 40% increase) and still runs in about a tenth of the 30-minute limit.

Risks accepted:

- The order of surcharge, weekend fee and discount is my assumption. If the spec owner decides otherwise, N1–N14 expected values must be recalculated.
- The invalid-zone defect and the rounding rule remain open.
- If the number of surcharges keeps growing, move to table-driven (data-driven) test generation, to avoid the combination count growing out of hand.